MANUALE DELLO SVILUPPATORE — SKILLSHARE

Cataudella Ilenia, 0001114156, ilenia.cataudella@studio.unibo.it
Dellaria Arianna, 0001125416, arianna.dellaria@studio.unibo.it
Zobbi Greta, 0001117792, greta.zobbi@studio.unibo.it

Requisiti **2**
Codice sorgente **2**
Avvio in modalità sviluppo **2**
Avvio con Docker **2**
Esecuzione dei test **3**
Struttura del progetto **3**
Come sono state realizzate le funzionalità **3**
Gestione Identità e Profilo 3
Gestione Annunci 4
Ricerca, filtri e ordinamento degli annunci 4
Conclusione di uno scambio e valutazione 5
Profilo pubblico dell'utente 5
Design pattern utilizzati **6**
Repository 6
Singleton (DatabaseCore) 6
Strategy 6
Persistenza dei dati **7**
Credenziali di test **7**

Requisiti

Per lavorare sul progetto o eseguirlo localmente sono necessari:

- **Java JDK 17+**: versione minima richiesta dal framework GWT utilizzato e dalle dipendenze del progetto.
- **Apache Maven**: gestisce dipendenze, compilazione e packaging dei tre moduli del progetto.
- **Docker Desktop**: necessario solo per l'avvio containerizzato, utile per verificare che l'applicazione funzioni in un ambiente isolato e riproducibile, indipendente dalla configurazione della macchina locale.

Codice sorgente

git clone https://github.com/AriannaDellaria/progetto-skillshare.git
cd [progetto-skillshare]/app

Tutti i comandi descritti nelle sezioni seguenti vanno eseguiti a partire dalla cartella app/, che contiene i tre moduli Maven del progetto.

Avvio in modalità sviluppo

Prima di un primo avvio, o dopo aver modificato le dipendenze nei file pom.xml, è necessario installare le dipendenze del progetto:

mvn clean install

L'avvio in modalità sviluppo richiede due processi eseguiti in parallelo, ciascuno in un terminale separato:

*mvn gwt:codeserver -pl *-client -am*

Il GWT Code Server compila il modulo **client** (le classi Gui) da Java a JavaScript e lo serve al browser, ricompilando automaticamente quando vengono rilevate modifiche al codice.

*mvn jetty:run -pl *-server -am -Denv=dev*

Il server Jetty espone il modulo **server** (le servlet RPC e la logica applicativa), gestendo le richieste provenienti dal client.

Una volta avviati entrambi i processi, l'applicazione risulta raggiungibile all'indirizzo http://localhost:8080/.

Avvio con Docker

L'avvio tramite Docker è possibile scrivendo nel terminale il seguente comando:

docker compose up -d --build

Successivamente, tramite Docker Desktop è possibile far partire l'applicazione cliccando sul link presente nella colonna *Port(s)*.

Esecuzione dei test

mvn test

Il comando esegue tutti i test unitari presenti nei moduli del progetto tramite JUnit 5. I test relativi alla logica applicativa (validazione, gestione utenti, gestione annunci) vengono eseguiti utilizzando un database in memoria, attivato tramite DatabaseCore.enableTestMode(), in modo da non interferire con il database persistente usato in fase di sviluppo o produzione.

Struttura del progetto

Il progetto è organizzato in tre moduli Maven ciascuno con una responsabilità distinta:

skillshare-client: contiene l'interfaccia utente. Ogni schermata dell'applicazione è implementata come una classe con suffisso Gui, che costruisce i widget dell'interfaccia e li rende visibili tramite il metodo mostra(). La classe App, dichiarata come entry point nel file module.gwt.xml, avvia l'applicazione mostrando la schermata di login.

skillshare-server: contiene la logica applicativa e l'esposizione dei servizi tramite RPC (Remote Procedure Call), il meccanismo di comunicazione tra client e server previsto da GWT. Ogni funzionalità è esposta tramite una classe con suffisso *ServiceImpl*, che riceve le richieste dal client, applica le regole di validazione e delega l'accesso ai dati alle classi *Repository*.

skillshare-shared: contiene il codice condiviso e compilabile sia lato client sia lato server, ovvero le classi dati, le interfacce dei servizi RPC (con le rispettive versioni sincrone e asincrone) e le classi di validazione, riutilizzabili in entrambi i contesti per evitare duplicazioni della logica di controllo.

Come sono state realizzate le funzionalità

Gestione Identità e Profilo

Il flusso di registrazione e accesso segue il seguente schema:

schermata (RegistrazioneGui, LoginGui) → servizio RPC (UtenteServiceImpl) → accesso ai dati (UtenteRepository).

In caso di errore in fase di registrazione (ad esempio nome utente già in uso, oppure campi non validi), il server restituisce un'eccezione contenente un messaggio descrittivo, che viene mostrato così com'è dal client all'interno del metodo onFailure della callback asincrona.

La sessione dell'utente autenticato viene mantenuta lato client dalla classe statica SessioneUtente, che conserva in memoria l'utente corrente per la durata dell'esecuzione della pagina. Le schermate che richiedono l'autenticazione verificano la presenza di un utente in sessione all'apertura, reindirizzando a LoginGui in sua assenza.

L'aggiornamento del profilo permette la modifica di bio, foto profilo e Tag di Competenza, mantenendo invariati nome utente ed email, che non sono modificabili.

Gestione Annunci

Le operazioni di creazione, modifica e rimozione degli annunci verificano lato server che l'utente richiedente sia effettivamente l'autore dell'annuncio interessato, controllo che non potrebbe essere garantito lato client. La rimozione di un annuncio non ne comporta la cancellazione fisica dal database, ma l'impostazione del campo attivo a false, escludendolo dagli elenchi pubblici pur mantenendone lo storico.

Ricerca, filtri e ordinamento degli annunci

La ricerca degli annunci tramite termini chiave e il filtro in base alla categoria o disponibilità utilizzano come accesso ai dati il metodo *cerca* della classe *AnnuncioRepository*, che, grazie ad un oggetto Matcher, individua gli annunci da inserire nella lista visualizzata dall'utente.
L'ordinamento viene effettuato sulla base della lista già filtrata e calcolato lato server, utilizzando un comparatore scelto in base all'input dell'utente.

Richiesta di scambio

Il server verifica, ad esempio, che l'annuncio esista, che non appartenga all'utente richiedente e che sia effettivamente disponibile, restituendo in caso contrario un'eccezione con messaggio descrittivo. Alla creazione, la richiesta viene associata allo stato iniziale "in attesa" e mantiene una copia del titolo dell'annuncio e del nome del mittente al momento dell'invio, evitando così ulteriori interrogazioni al server in fase di visualizzazione.

L'utente può consultare in ogni momento l'esito delle proprie richieste, così come il proprietario di un annuncio può consultare le richieste ricevute, entrambe recuperate tramite metodi dedicati di *RichiestaScambioRepository* che filtrano le richieste rispettivamente per mittente e per proprietario.

Gestione delle richieste ricevute

Dalla schermata delle richieste ricevute, il proprietario dell'annuncio può accettare o rifiutare ciascuna richiesta ancora in stato "in attesa", previa conferma esplicita richiesta all'utente. Il server verifica, prima di modificare lo stato della richiesta, che l'operazione sia richiesta effettivamente dal proprietario dell'annuncio e che la richiesta non sia già stata gestita in precedenza.

Se l'utente elimina un annuncio per cui sono presenti richieste ancora in attesa, queste vengono automaticamente impostate allo stato "rifiutata". Questo meccanismo si applica solo alla rimozione dell'annuncio, non al semplice cambio di disponibilità.

Chat dello scambio

Quando una richiesta viene accettata, i due partecipanti accedono a una conversazione dedicata, identificata dalla richiesta di scambio a cui è associata. Prima di mostrare la cronologia o permettere l'invio di un messaggio, il server verifica che l'utente richiedente sia

effettivamente uno dei due partecipanti coinvolti e che lo scambio si trovi nello stato "accettata", negando altrimenti l'accesso. I messaggi vengono ordinati cronologicamente al momento della visualizzazione.

La condivisione di un link, riservata a chi ha pubblicato l'annuncio, viene validata separatamente rispetto ai messaggi testuali tramite la classe ValidatoreLink, che verifica la correttezza del formato prima del salvataggio; un link non valido non viene registrato e l'utente riceve un messaggio d'errore descrittivo.

Conclusione di uno scambio e valutazione

Una richiesta di scambio può avere quattro stati: in attesa, accettata, rifiutata e conclusa.
Il passaggio dallo stato “accettata” a “conclusa” viene gestito nella classe *RichiestaScambioServiceImpl* ed è consentito solo al proprietario dell’annuncio. Nella fase di chiusura, viene gestita anche la chiusura della chat associata allo scambio tramite un controllo sullo stato dello scambio stesso.
Terminato uno scambio, è possibile assegnare una valutazione all’altro utente coinvolto (sia numerica, che tramite commento); il destinatario e il tipo di valutazione (prestazione o controprestazione) vengono dedotti dal server in base al ruolo che si ricopriva in quello scambio. Viene fatto anche un controllo rispetto al numero di valutazioni per scambio, infatti possono esistere massimo due valutazioni per scambio, una per direzione.

Profilo pubblico dell’utente

Ogni utente ha un proprio profilo pubblico, reso accessibile tramite un link nella pagina di esplorazione degli annunci e nella pagina delle richieste ricevute. Nel profilo pubblico sono mostrati rating e storico, le informazioni vengono recuperate lato server dai metodi *calcolaRating* e *recuperaStorico* di *ValutazioneServiceImpl*. Il rating viene calcolato come la media di tutte le valutazioni numeriche ottenute, distinte per tipo; viene mostrato sia un rating relativo alle prestazioni, che uno relativo alle controprestazioni. Lo storico viene anch’esso diviso per prestazione e controprestazione, mostrando per ogni scambio titolo dell’annuncio o testo della controprestazione.

Design pattern utilizzati

Repository

Separano la logica di accesso ai dati, ovvero le interrogazioni sulla struttura MapDB, dalla logica di business, contenuta invece nelle classi *ServiceImpl*. Ogni classe *Repository* espone metodi con nomi descrittivi del dominio applicativo (es. *trovaPerEmail*, *trovaAttivi*, *modificaAnnuncio*) anziché esporre direttamente la struttura dati sottostante.

Vantaggio: un eventuale cambiamento del meccanismo di persistenza, ad esempio il passaggio a un database relazionale, richiederebbe modifiche contenute alle sole classi Repository, lasciando invariate le servlet e l'interfaccia utente. Il pattern semplifica inoltre la scrittura di test mirati sulla sola logica di business, mantenendo la logica di accesso ai dati isolata e testabile separatamente.

Singleton

Garantisce l'esistenza di un'unica connessione attiva al database MapDB per l'intera applicazione, condivisa da tutte le classi Repository tramite il metodo statico *getDB()*.

Vantaggio: previene i conflitti derivanti da connessioni concorrenti allo stesso file di database, evitando possibili corruzioni dei dati. Centralizza inoltre la gestione della modalità di test, tramite i metodi *enableTestMode()* e *disableTestMode()*, che permettono di sostituire il database su file con un database temporaneo in memoria durante l'esecuzione dei test automatizzati.

Strategy

Il pattern comportamentale strategy viene utilizzato per separare un oggetto dal suo comportamento, in Skillshare viene utilizzato per gestire la ricerca e il filtraggio degli annunci. È stata definita l'interfaccia *AnnuncioMatcher* con il metodo *matches(Annuncio)*, il metodo viene implementato in due diverse classi:

- *FiltroAnnuncioMatcher*: per individuare la categoria e le disponibilità da filtrare;
- *KeywordAnnuncioMatcher*: cerca la parola chiave nel titolo o nella categoria dell'annuncio.

Le due strategie vengono utilizzate da *AnnuncioRepository* nel metodo *cerca()*.

Vantaggio: il Repository resta invariato anche all'introduzione di nuovi criteri di ricerca, che per essere aggiunti richiederanno solo la creazione di nuove classi e non la modifica di altre già esistenti.

Persistenza dei dati

Il database MapDB viene salvato nel file app/progetto_sweng.db. Si tratta di un database embedded, ovvero eseguito all'interno della stessa JVM dell'applicazione, senza necessità di un server esterno, e strutturato secondo l'interfaccia standard di java.util.Map.

Il database contiene:

utenti: mappa che associa l'identificativo univoco di ciascun utente (id, generato tramite UUID) al relativo oggetto *Utente*.

annunci: mappa che associa l'identificativo univoco di ciascun annuncio (id, generato tramite UUID) al relativo oggetto *Annuncio*.

richiesteScambio: mappa che associa l'identificativo univoco di ciascuna richiesta di scambio (id, generato tramite UUID) al relativo oggetto *RichiestaScambio*, insieme allo stato corrente.

valutazioni: mappa che associa l'identificativo univoco di ciascuna valutazione (id, generato tramite UUID) al relativo oggetto *Valutazione*, con punteggio, commento e tipo.

messaggi: mappa che associa l'identificativo univoco di ciascuna valutazione (id, generato tramite UUID) al relativo oggetto *Messaggio*.

Ogni operazione di scrittura (creazione, modifica, rimozione logica) viene seguita da una chiamata a *DatabaseCore.commit()*, necessaria per rendere permanenti su disco le modifiche apportate alla struttura in memoria.

Credenziali di test

Nome utente	**Password**
Ilenia	Ilenia123!
Arianna	Arianna123!
Greta	Greta123!